第9章 数据库完整性和安全性

西安交通大学计算机科学与技术学院　　任雪斌

xuebinren@mail.xjtu.edu.cn

目录

9.1 引言

9.2 数据库的完整性

9.3 数据库的安全性

9.1 引言

➢引言

- 数据库是长期保存在计算机内的有组织的可共享数据集合
- 在投入运行后，一方面需要利用数据库系统统一管理和共享数据
- 另一方面需要提供必要的措施保证其中数据的完整性和安全性

9.1 引言

➢引言

- 数据库中发生数据错误，归纳起来主要有以下四个方面的原因：
  - ✓系统发生软、硬件故障，数据遭到破坏
  - ✓事务并发执行引起数据的不一致性
  - ✓数据库的更新操作有误，如输入本身是错误的数 据、不正确的操作或程序产生的不一致数据等。
  - ✓自然的或人为的破坏，例如意外事故（失火、失窃等）、恶意的攻击篡改数据等等

➢引言

- 针对这四种情况，DBMS必须提供以下几项数据控制功能：
  - ✓数据库恢复：将数据库从错误状态恢复到某一已知的一致状态的功能
  - ✓并发控制：协调并发事务的执行，并维护数据的一致性
  - ✓保持完整性：数据库中数据始终保证是正确的和一致的
  - ✓保证安全性：防止非法使用数据库，造成数据泄露、篡改和破坏

9.2 数据库完整性

➢数据库完整性

- 数据库的完整性是指数据的正确性、有效性和相容性
- 数据库是否具备完整性关系到数据库系统能否真实地反映现实世界，因此维护数据库的完整性是非常重要的
- 为维护数据库的完整性，DBMS必须提供一种机制来检查数据库中的数据，看其是否满足语义规定的条件

9.2 数据库完整性

➢完整性约束条件

- 完整性检查是围绕完整性约束条件进行的，因此，完整性约束条件是完整性控制机制的核心
- 我们将加在数据库之上的语义约束条件称为数据库完整性约束条件，它们作为模式的一部分存入数据库中
- 而DBMS中检查数据是否满足完整性条件的机制称为完整性检查

9.2 数据库完整性

➢完整性约束条件

- 完整性约束条件作用的对象可以是关系、元组、列三种
- 其中列约束主要是列的类型、取值范围、精度、排序等的约束条件
- 元组的约束是元组中各个字段间的联系的约束
- 关系的约束是若干元组间、关系集合上以及关系之间的联系的约束

9.2 数据库完整性

➢完整性约束条件

- 完整性约束条件涉及的这三类对象，其状态可以是静态的，也可以是动态的
- 所谓静态约束是指数据库每一确定状态时的数据对象所应满足的约束条件，它是反映数据库状态合理性的约束，这是最重要的一类完整性约束
- 动态约束是指数据库从一种状态转变为另一种状态时新、旧值之间所应满足的约束条件，它是反映数据库状态变迁的约束
- 综合上述两个方面，我们可以将完整性约束条件分为6类

9.2 数据库完整性

➢完整性约束条件

- 静态列级约束：是对一个列的取值域的说明，这是最常用也最容易实现的一类完整性约束，包括以下几方面：
  1) 对数据类型的约束，包括数据的类型、长度、单位、精度等；
  2) 对数据格式的约束；
  3) 对取值范围或取值集合的约束；
  4) 对空值(NULL)的约束;
  5) 其他约束

9.2 数据库完整性

➢完整性约束条件

- 静态元组约束：一个元组是由若干个列值组成的，静态元组约束就是规定元组的各个列之间的约束关系

9.2 数据库完整性

➢完整性约束条件

- 静态关系约束： 在一个关系的各个元组之间或者若干关系之间常常存在各种联系或约束。常见的静态关系约束有：

  1) 实体完整性约束；
  2) 参照完整性约束；实体完整性约束和参照完整性约束是关系模型的两个极其重要的约束，称为关系的两个不变性。
  3) 函数依赖约束。大部分函数依赖约束都在关系模式中定义。
  4) 统计约束。即字段值与关系中多个元组的统计值之间的约束关系。

9.2 数据库完整性

➢完整性约束条件

- 动态列级约束：是修改列定义或列值时应满足的约束条件，包括下面两方面：

1） 修改列定义时的约束：

例如，将允许空值的列改为不允许空值时，如果该列目前已存在空值，则拒绝这种修改

1） 修改列值时的约束：

修改列值有时需要参照其旧值，并且新旧值之间需要满足某种约束条件。例如，职工工资调整不得低于其原来工资，学生年龄只能增长等等

9.2 数据库完整性

➢完整性约束条件

- 动态元组约束：动态元组约束是指修改元组的值时元组中各个字段间需要满足某种约束条件。

  例如职工工资调整时新工资不得低于 原工资+工龄*5，等等。

- 动态关系约束：动态关系约束是加在关系变化前后状态上的限制条件，例如事务一致性、原子性等约束条件

9.2 数据库完整性

➢完整性控制

- DBMS的完整性控制机制应具有三个方面的功能：

1) 定义功能，提供定义完整性约束条件的机制
2) 检查功能，检查用户发出的操作请求是否违背了完整性约束条件
3) 如果发现用户的操作请求使数据违背了完整性约束条件，则采取一定的动作来保证数据的完整性

9.2 数据库完整性

➢完整性控制

- 检查是否违背完整性约束的时机通常是在一条语句执行完后立即检查，我们称这类约束为立即执行约束（**Immediate constraints**）
- 有时完整性检查需要延迟到整个事务执行结束后再进行，检查正确方可提交，我们称这类约束为延迟执行约束（**Deferred constraints**）
- 在关系系统中，最重要的完整性约束是实体完整性和参照完整性，其他完整性约束条件可以归入用户定义的完整性
- 下面详细讨论实现参照完整性要考虑的几个问题

9.2 数据库完整性

➢完整性控制

- 外键能否接受空值问题

  1)　在实现参照完整性时，系统除了应该提供定义外键的机制，还应提供定义外键列是否允许空值的机制。

- 在被引用关系中删除元组的问题

  一般地，当删除被引用关系的某个元组，而引用关系存在若干元组，其外键值与被引用关系删除元组的主键值相同，这时可有三种不同的策略：

  (1) 级联删除（CASCADE）

  (2) 受限删除（RESTRICT）：仅当引用关系中没有任何元组的外键值与被引用关系中要删除元组的主键值相同时，系统才执行删除操作，否则拒绝此删除操作。

  (3) 置空值删除（SET NULL）：删除被引用关系的元组，并将引用关系中相应元组的外键值置空值。

9.2 数据库完整性

➢完整性控制

- 例如要删除被引用关系Student中学号等于04055001的学生元组，而在引用关系SC中存在学号等于04055001的选课元组。

  (1) 级联删除（CASCADE）

  (2) 受限删除（RESTRICT）

  (3) 置空值删除（SET NULL）

9.2 数据库完整性

➢完整性控制

- 在引用关系中插入元组时的问题

一般地，当引用关系插入某个元组，而被引用关系不存在相应的元组，其主键值与引用关系插入元组的外键值相同，这时可有以下策略：

（1）受限插入

（2）递归插入

9.2 数据库完整性

➢完整性控制

- 例如将一个新的选课元组（学号：04055001；课程号：CS-01；成绩：88）插入引用关系SC中，而在被引用关系Student中暂时不存在学号等于04055001的学生元组。

  (1) CASCADE

  (2) RESTRICT

9.2 数据库完整性

➢完整性控制

修改关系中主键的问题

a) 不允许修改主键

b) 允许修改主键

- 在有些RDBMS中，允许修改关系主键，但必须保证主键的唯一性和非空，否则拒绝修改 （CASCADE、RESTRICT、SET NULL)

✓从上面的讨论我们看到DBMS在实现参照完整性时，除了要提供定义主键、外键的机制外，还需要提供不同的策略供用户选择。选择哪种策略，都要根据应用环境的要求确定

9.2 数据库完整性

➢完整性约束的说明

- 完整性约束的说明一般分为隐含约束和显式约束两类
- 隐含约束的说明可用于实现域完整性约束、实体完整性约束以及参照完整性约束等，如定义主关键字、外关键字；在设计关系数据模式时，还可定义属性的类型、长度、单位、精度等
- DBMS根据这些定义对数据库的更新操作自动检查，维护数据的完整性，这是数据库中最基本、最普遍的约束

9.2 数据库完整性

➢完整性约束的说明

- 显式约束的说明可实现一些隐含约束无法表达的约束，这些约束依赖于数据的语义和应用，本身比较复杂
- 用户根据不同DBMS的支持程度，一般选择使用4种可能的方法

9.2 数据库完整性

➢完整性约束的说明

- CHECK子句

✓在基表定义中，加入一个CHECK子句，利用此子句说明各列的值应满足的约束条件

✓这种约束是对单个关系的元组值进行约束，条件中也可以涉及本关系的其它元组或其它关系的元组

✓此时，CHECK子句只对定义它的关系起约束作用，对于条件中提到的其它关系没有约束作用

9.2 数据库完整性

➢完整性约束的说明

- 断言

✓每个断言就是一个谓词，指定了数据库状态必须满足的逻辑条件

✓利用断言表达数据库完整性约束，形成一系列断言的集合

✓DBMS对每个更新事务，用集合中的断言对它进行检查。只有不破坏断言的更新操作才允许，否则，就撤销事务

9.2 数据库完整性

➢完整性约束的说明

- 断言

✓SQL中定义断言用CREATE语句，形式如下：

CREATE ASSERTION <断言名>

CHECK <逻辑条件>;

✓这里<逻辑条件>和SQL中查询语句的查询条件一样

✓撤销断言用DROP语句，形式如下：

DROP ASSERTION <断言名>;

9.2 数据库完整性

➢完整性约束的说明

- 断言

例：在产品定单(Order)中，可以通过下面的断言限定其销售价格(sale_price)不得小于相关产品报价(list_price)的80%：

```
CREATE  ASSERTION  price-constraint CHECK (
    NOT  EXISTS (
        SELECT  *   FROM   Order O
        WHERE  sale_price  <  ( SELECT 0.8 * list_price
            FROM Product P WHERE P.PNO=O.PNO    )
    )
);
```

9.2 数据库完整性

➢完整性约束的说明

- 断言

✓断言的引入减少了编程的麻烦，本身也比较容易维护

✓但是如果断言较复杂，则给创建和检查会带来相当大的开销，同时降低了数据库更新操作的性能

✓因此，对于断言的使用应该权衡考虑

9.2 数据库完整性

➢完整性约束的说明

- 存储过程

✓存储过程是指将常用的访问数据库的程序，作为一个过程进行编译，然后存储在数据库中，供用户调用

✓这样做既改善性能，又方便用户，还扩充了功能

✓将存储过程应用到完整性约束，就是在应用程序中插入一些存储过程说明完整性约束，在更新数据库时执行，完成数据库完整性的检查

✓如果违反给定的约束，则撤销事务，或者修改数据使数据库保持完整性

9.2 数据库完整性

➢完整性约束的说明

- 触发器

✓触发器(trigger)是一条在对数据库执行更新操作时系统自动执行的语句，又称主动规则或ECA（事件 - 条件 - 动作）规则

✓即当发生某一事件时，如果满足给定的条件，则执行相应的动作

✓利用触发器可以表示约束，以某种更新数据库的操作作为事件，以违反约束作为条件，以违反约束的处理作为动作

9.2 数据库完整性

➢完整性约束的说明

- 触发器

✓在插入、删除、修改等更新操作发生时，触发器开始工作

✓先检查完整性约束条件是否满足，如果满足就执行相应的动作

✓动作可以有很多形式，如撤销事务，或传递一个消息给用户，或完成某些对数据库的操作，以保持数据库的完整性

✓触发器的用途不止这一点，它还可以用在监视数据库的状态等

9.2 数据库完整性

➢完整性约束的说明

- 触发器

例：在产品定单(Order)中，也可以通过下面的触发器限定其销售价格(sale_price)不得小于相关产品报价(list_price)的80%：

```
CREATE  TRIGGER price-constraint
    BEFORE  INSERT  ON  Order
        REFERENCING  NEW  AS  N
            FOR  EACH  ROW
                WHEN (N. sale_price<
                    (SELECT 0.8 * list_price  FROM Product P WHERE P.PNO=N.PNO )
ROLLBACK;
```

9.3 数据库系统安全性

➢数据库系统安全性

- 计算机系统安全性，是指为保护计算机系统中的硬件、软件及数据，防止其因偶然或恶意的原因遭到破坏、篡改或泄露等，而采取的措施；
- 计算机安全不仅涉及到计算机系统本身的技术问题，还涉及管理学、法学、犯罪学、心理学的问题等方面
- 其内容包括计算机安全理论与策略；计算机安全技术、安全管理、安全评价、安全产品以及计算机犯罪与侦察、计算机安全法律、安全监察等等
- 概括起来，计算机系统的安全性问题可分为三大类，即：技术安全类、管理安全类和政策法律类

9.3 数据库系统安全性

➢数据库系统安全性

- 技术安全性

是指计算机系统中采用具有一定安全性的硬件、软件来实现对计算机系统及其所存数据的安全保护，当计算机系统受到无意或恶意的攻击时仍能保证系统正常运行，保证系统内的数据不增加、不丢失、不泄露。

- 管理安全性

技术安全之外的，诸如软硬件意外故障、场地的意外事故、管理不善导致的计算机设备和数据介质的物理破坏、丢失等安全问题，视为管理安全

- 政策法规

则指政府部门建立的有关计算机犯罪、数据安全保密的法律道德准则和政策法规、法令等。

- **本课程只讨论技术安全类**

9.3 数据库系统安全性

➢计算机系统安全模型

- 在一般计算机系统中，安全措施是一级一级层层设置的。例如可以有如下的模型：

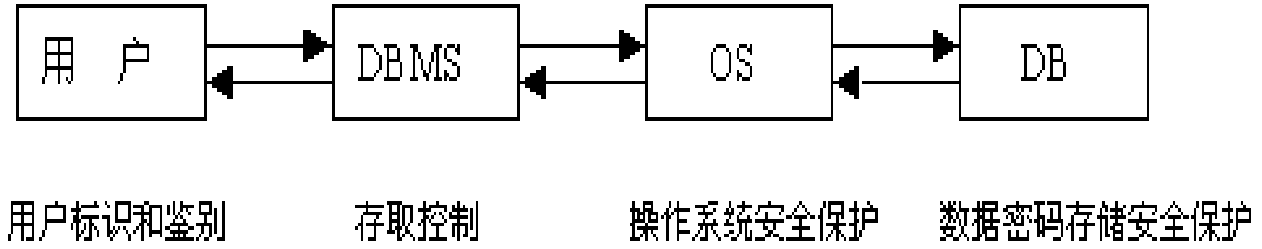


图 9.1 计算机系统的安全模型

9.3 数据库系统安全性

➢用户标识和鉴别

- 用户标识和鉴别是系统提供的最外层安全保护措施
- 其方法是由系统提供一定的方式让用户标识自己的名字或身份
- 每次用户要求进入系统时，由系统进行核对，通过鉴定后才提供机器使用权
- 例如，使用用户名和口令

9.3 数据库系统安全性

➢数据库系统的存取控制机制

- 存取控制机制主要包括两部分：
  - ✓ 定义用户权限，并将用户权限登记到数据目录中
  - ✓ 合法权限检查，每当用户发出存取数据库的操作请求后（请求一般应包括操作类型、操作对象和操作用户等信息），DBMS查找数据目录，根据安全规则进行合法权限检查，若用户的操作请求超出了定义的权限，系统将拒绝执行此操作
- 用户权限定义和合法权检查机制一起组成了DBMS的安全子系统

9.3 数据库系统安全性

➢数据库系统的存取控制机制

- 当前大型的DBMS一般都支持C2级中的自主存取控制 （DAC）
- 有些DBMS同时还支持B1级中的强制存取控制(MAC)
- 在强制存取控制中，每一个数据对象被标以一定的“密级”，每一个用户也被授予某一个级别的“许可证”
- 对于任意一个对象，只有具有合法许可证的用户才可以存取
- 因此，强制存取控制相对自主存取控制更加严格

9.3 数据库系统安全性

➢自主存取控制

- 大型数据库管理系统几乎都支持自主存取控制，目前的SQL标准也对自主存取控制提供支持，这主要通过SQL 的GRANT语句和REVOKE语句来实现
- 用户权限是由两个要素组成的：数据对象和操作类型
- 定义一个用户的存取权限就是要定义这个用户可以在哪些数据对象上进行哪些类型的操作

9.3 数据库系统安全性

➢自主存取控制

- 在SQL中，有两种授权的方式。一种是在数据库一级，由DBA负责授予和收回用户需要得到的必要特权，只有取得这种授权，才能成为数据库用户；另一种是把自身拥有的权限转授给其他用户的授权，可以由DBA或者数据对象的创建者授予对某些数据对象进行某些操作的特权。

9.3 数据库系统安全性

➢自主存取控制

- 第一种授权使用GRANT语句的语句格式如下
- GRANT<特权类型>[{，<特权类型〉}] TO <用户名>[IDENTIFIED BY <口令>]；
- 收回权限使用REVOKE语句的语句格式如下
- REVOKE<特权类型>[{，<特权类型〉}] FROM <用户名>；
- <特权类型> ::=CONNECT|RESOURCE|DBA

9.3 数据库系统安全性

➢自主存取控制

- 例9-1 授予用户USER1在数据库上的CONNECT特权。
- GRANT CONNECT TO USER1 IDENTIFIED BY 1234;
- 收回用户USER1在数据库上的RESOURCE特权。
- REVOKE RESOURCE FROM USER1;

9.3 数据库系统安全性

➢自主存取控制

- 第二种授权使用GRANT语句的语句格式如下。
- GRANT<特权>ON<数据对象>TO<受权者>[{,<受权者>}][WITH GRANT OPTION];
- <特权>::=ALL PRIVILIEGES|<操作>[{,<操作>}]
- <操作>::=SELECT|INSERT|UPDATE|DELETE[(<准许修改的属性表>)]
- <准许修改的属性表>::=<属性名>[{,<属性名>}]
- <受权者>::=PUBLIC|<用户名>
- 收回特权可用下面的REVOKE语句:
- REVOKE<特权>ON<数据对象>FROM<受权者>[{,<受权者>}][RESTRICT|CASCADE];

9.3 数据库系统安全性

➢自主存取控制

- 例9-2 授予用户USER1在Shop表上的SELECT和INSERT特权。
- GRANT SELECT,INSERT ON TABLE Shop TO USER1;
- 授予用户USER1在Customer表上的所有特权。
- GRANT ALL PRIVILIEGES ON TABLE Customer TO USER1;
- 授予用户USER1、USER2在SC表上的SELECT特权和对属性Item、Quantity的UPDATE权限。
- GRANT SELECT,UPDATE(Item,Quantity) ON TABLE SC TO USER1,USER2;

9.3 数据库系统安全性

➢自主存取控制

- 收回前面所授予的权限。
- REVOKE SELECT,INSERT ON TABLE Shop FROM USER1;
- REVOKE ALL PRIVILIEGES ON TABLE Customer FROM USER1;
- REVOKE SELECT, UPDATE (Item,Quantity) ON TABLE SC FROM USER1,USER2;

9.3 数据库系统安全性

➢自主存取控制

- 在数据库系统中，定义存取权限称为授权(Authorization)
- 用户权限定义中数据对象范围越小授权子系统就越灵活。
- 例如，上面的授权定义可精细到字段级，而有的系统只能对关系授权。授权粒度越细，授权子系统就越灵活，但系统定义与检查权限的开销也会相应地增大

9.3 数据库系统安全性

➢自主存取控制

- 衡量授权子系统精巧程度的另一个尺度是能否提供与数据值有关的授权
- 如上面的授权定义是独立于数据值的，即用户能否对某类数据对象执行的操作与数据值无关，完全由数据名决定
- 反之，若授权依赖于数据对象的内容，则称为是与数据值有关的授权。
- 有的系统还允许存取谓词中引用系统变量，如一天中的某个时刻，某台终端设备号,这样用户只能在某台终端、某段时间内存取有关数据，这就是与时间和地点有关的存取权限

9.3 数据库系统安全性

➢自主存取控制

- 自主存取控制能够通过授权机制有效地控制其他用户对敏感数据的存取
- 但是由于用户对数据的存取权限是“自主”的，用户可以自由地决定将数据的存取权限授予何人、决定是否也将“授权”的权限授予别人
- 在这种授权机制下，仍可能存在数据的“无意泄露”

9.3 数据库系统安全性

➢强制存取控制

- 所谓强制存取控制是指系统为保证更高程度的安全性，按照TDI/TCSEC标准中安全策略的要求，所采取的强制存取检查手段
- 强制存取控制不是用户能直接感知或进行控制的
- 强制存取控制适用于那些对数据有严格而固定密级分类的部门，例如军事部门或政府部门
- 在强制存取控制中，DBMS所管理的全部实体被分为主体和客体两大类

9.3 数据库系统安全性

➢强制存取控制

- 主体是系统中的活动实体，既包括DBMS所管理的实际用户，也包括代表用户的各进程
- 客体是系统中的被动实体，是受主体操纵的，包括文件、基表、索引、视图等等
- 对于主体和客体，DBMS为它们每个实例（值）指派一个敏感度标记（Label）

9.3 数据库系统安全性

➢强制存取控制

- 敏感度标记被分成若干级别，例如绝密(Top Secret)、机密(Secret)、可信(Confidential)、公开(Public)等
- 主体的敏感度标记称为许可证级别(Clearance Level)，客体的敏感度标记称为密级（Classification Level）
- MAC机制就是通过对比主体的Label和客体的Label，最终确定主体是否能够存取客体

9.3 数据库系统安全性

➢强制存取控制

- 当某一用户（或一主体）以标记label注册入系统时，系统要求他对任何客体的存取必须遵循如下规则：

✓ 仅当主体的许可证级别大于或等于客体的密级时，该主体才能读取相应的客体

✓ 仅当主体的许可证级别等于客体的密级时，该主体才能写相应的客体

9.3 数据库系统安全性

➢强制存取控制

- 规则(1)的意义是明显的。而规则(2)需要解释一下

✓ 在某些系统中，第(2)条规则与这里的规则有些差别。这些系统规定：仅当主体的许可证级别小于或等于客体的密级时，该主体才能写相应的客体，即用户可以为写入的数据对象赋予高于自己的许可证级别的密级

✓ 这样一旦数据被写入，该用户自己也不能再读该数据对象了

✓ 这两种规则的共同点在于它们均禁止了拥有高许可证级别的主体更新低密级的数据对象，从而防止了敏感数据的泄漏

9.3 数据库系统安全性

➢强制存取控制

- 强制存取控制(MAC)是对数据本身进行密级标记，无论数据如何复制，标记与数据是一个不可分的整体，只有符合密级标记要求的用户才可以操纵数据，从而提供了更高级别的安全性
- 前面已经提到，较高安全性级别提供的安全保护要包含较低级别的所有保护，因此在实现MAC时要首先实现DAC，即DAC与MAC共同构成DBMS的安全机制
- 系统首先进行DAC检查，对通过DAC检查的允许存取的数据对象再由系统自动进行MAC检查，只有通过MAC检查的数据对象方可存取

9.3 数据库系统安全性

➢视图机制

- 进行存取权限控制时我们可以为不同的用户定义不同的视图，把数据对象限制在一定的范围内，也就是说，通过视图机制把要保密的数据对无权存取的用户隐藏起来，从而自动地对数据提供一定程度的安全保护
- 视图机制间接地实现了支持存取谓词的用户权限定义
- 在不直接支持存取谓词的系统中，可以先建立视图，然后在视图上进一步定义存取权限

9.3 数据库系统安全性

➢审计

- 前面讲的用户标识与鉴别、存取控制仅是安全性标准的一个重要方面，但不是全部
- 因为，任何系统的安全保护措施都不是完美无缺的，蓄意盗窃、破坏数据的人总是想方设法打破控制
- 为了使DBMS达到一定的安全级别，还需要在其它方面提供相应的支持
- 例如按照TDI/TCSEC标准中安全策略的要求，“审计”功能就是DBMS达到C2以上安全级别必不可少的一项指标

9.3 数据库系统安全性

➢审计

- 审计功能把用户对数据库的所有操作自动记录下来放入审计日志（Audit Log）中
- DBA可以利用审计跟踪的信息，重现导致数据库现有状况的一系列事件，找出非法存取数据的人、时间和内容等
- 审计通常是很费时间和空间的，所以DBMS往往都将其作为可选特征，允许DBA根据应用对安全性的要求，灵活地打开或关闭审计功能
- 审计功能一般主要用于安全性要求较高的部门

9.3 数据库系统安全性

➢数据加密

- 对于高度敏感性数据，例如财务数据、军事数据、国家机密，除以上安全性措施外，还可以采用数据加密技术
- 数据加密是防止数据库中数据在存储和传输中失密的有效手段
- 加密的基本思想是根据一定的算法将原始数据(术语为明文，Plain text)变换为不可直接识别的格式(术语为密文，Cipher text)，从而使得不知道解密算法的人无法获知数据的内容

9.3 数据库系统安全性

➢数据加密

- 加密方法主要有两种，一种是替换方法，该方法使用密钥（Encryption Key）将明文中的每一个字符转换为密文中的一个字符
- 另一种是置换方法，该方法仅将明文的字符按不同的顺序重新排列。单独使用这两种方法的任意一种都是不够安全的
- 但是将这两种方法结合起来就能提供相当高的安全程度。采用这种结合算法的例子是美国1977年制定的官方加密标准，数据加密标准（Data Encryption Standard，简称DES）
- 有关DES秘密密钥加密技术及密钥管理问题等已超出本课程的范围，这里不再讨论

9.4数据库系统隐私性

➢统计数据库(statistical database)：

- 用户仅可针对数据的统计信息进行聚集查询（count, mean, sum等等）
- 不能对单条记录进行查询

“对统计数据库的访问应当使任何人不能获得个体数据的信息”[1][2]

Name	Has cancer?
Alice	Yes
Bob	No
Chris	Yes
Denise	Yes
Eric	No
Frank	Yes

[1] T. Dalenius, Towards a methodology for statistical disclosure control. Statistik Tidskrift 15, pp. 429–222, 1977.
[2] S. Goldwasser and S. Micali, Probabilistic encryption. Journal of Computer and System Sciences 28, pp. 270–299, 1984; preliminary version appeared in Proceedings 14th Annual ACM Symposium on Theory of Computing, 1982.

9.4数据库系统隐私性

➢差分隐私

- 通过统计学定义严格限制计算结果中对数据的信息披露程度

统计数据库从本质上无法保证隐私！



Cynthia Dwork

9.4数据库系统隐私性

➢差分隐私

- 通过统计学定义严格限制计算结果中对数据的信息披露程度

查询1：4个人患有癌症

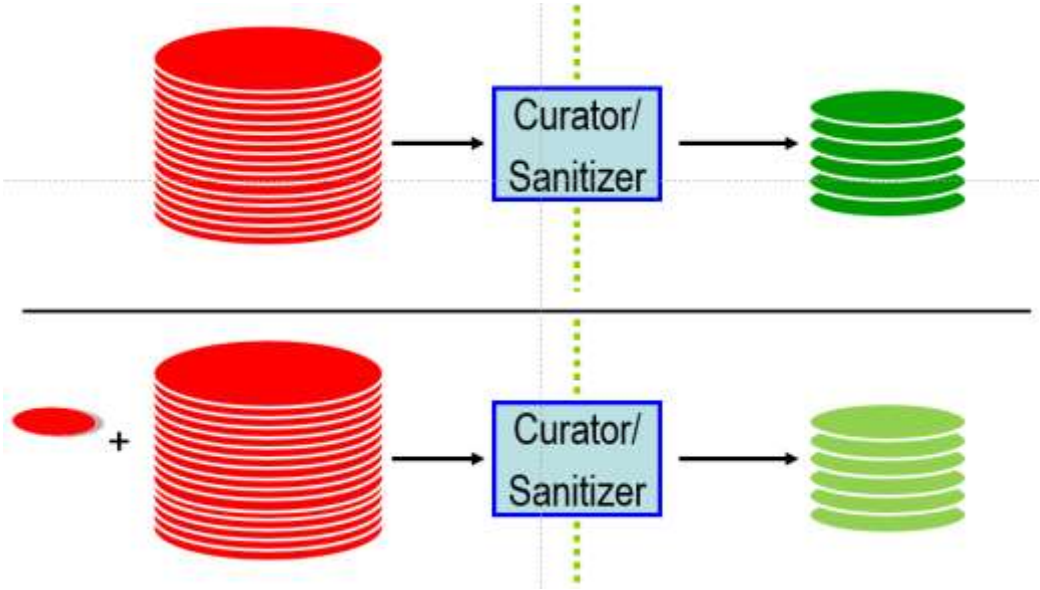
Name	Has cancer?
Alice	Yes
Bob	No
Chris	Yes
Denise	Yes
Eric	No
Frank	Yes

查询2：3个人患有癌症

9.4数据库系统隐私性

➢差分隐私

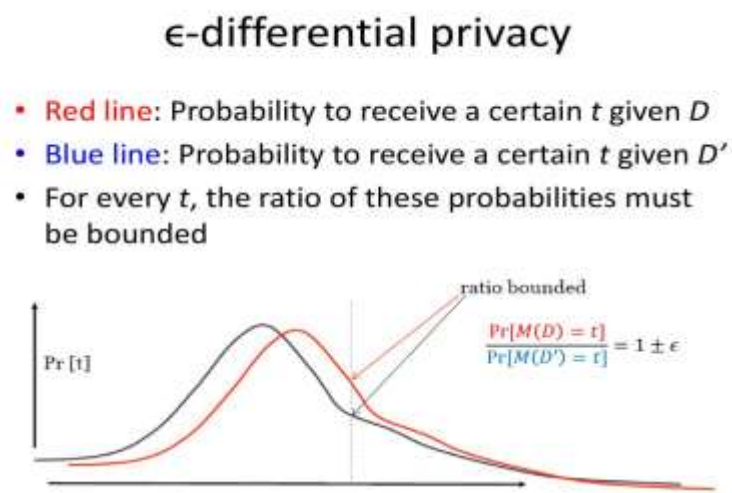
- 通过统计学定义严格限制计算结果中对数据的信息披露程度
  - 与攻击者背景知识无关
  - 两个输入数据集仅相差一条记录，输出结果非常非常接近，攻击者难以区分



9.4数据库系统隐私性

➢差分隐私定义

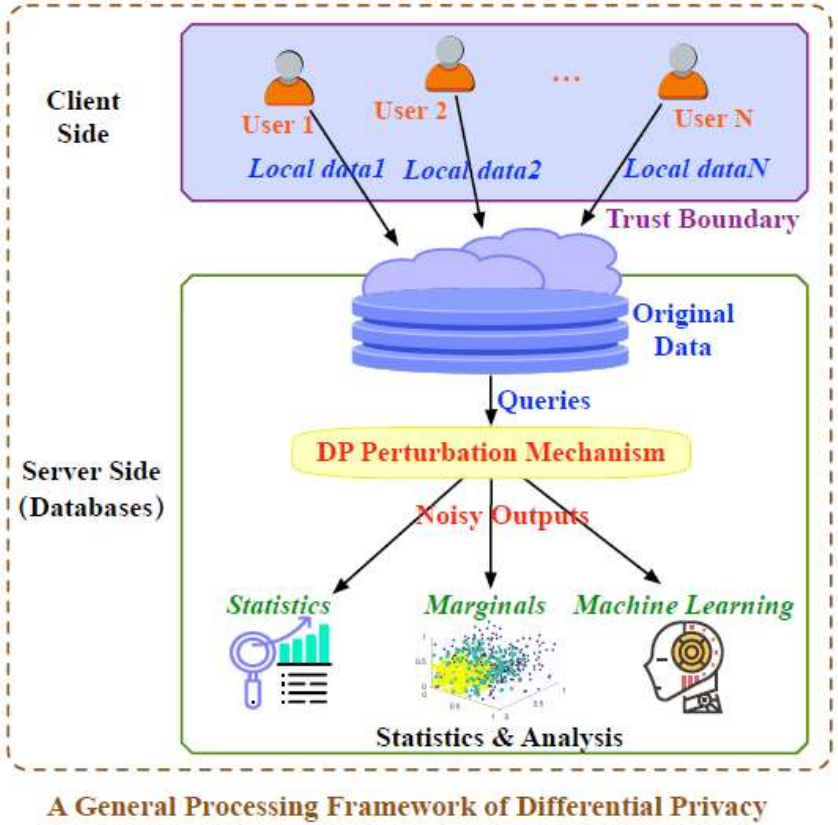
- 给定一个随机算法M，如果其对于任意的相邻数据集$\mathcal{D}$和$\mathcal{D}'$，以及M的任何输出结果范围S S⊆Range (M)，均有 $P(M(\mathcal{D}) \in S) \leq e\text{^}\varepsilon \cdot P(M(\mathcal{D}') \in S)$, 则称M满足ε-差分隐私。
- 其中，相邻集$\mathcal{D}$和$\mathcal{D}'$表示仅仅相差了一条记录的两个数据集，ε表示隐私预算， ε越小隐私保护程度越高。
- 差分隐私可以通过添加噪声实现，使相差一条记录的两个数据集$\mathcal{D}$和$\mathcal{D}'$上输出同一结果的概率相近。



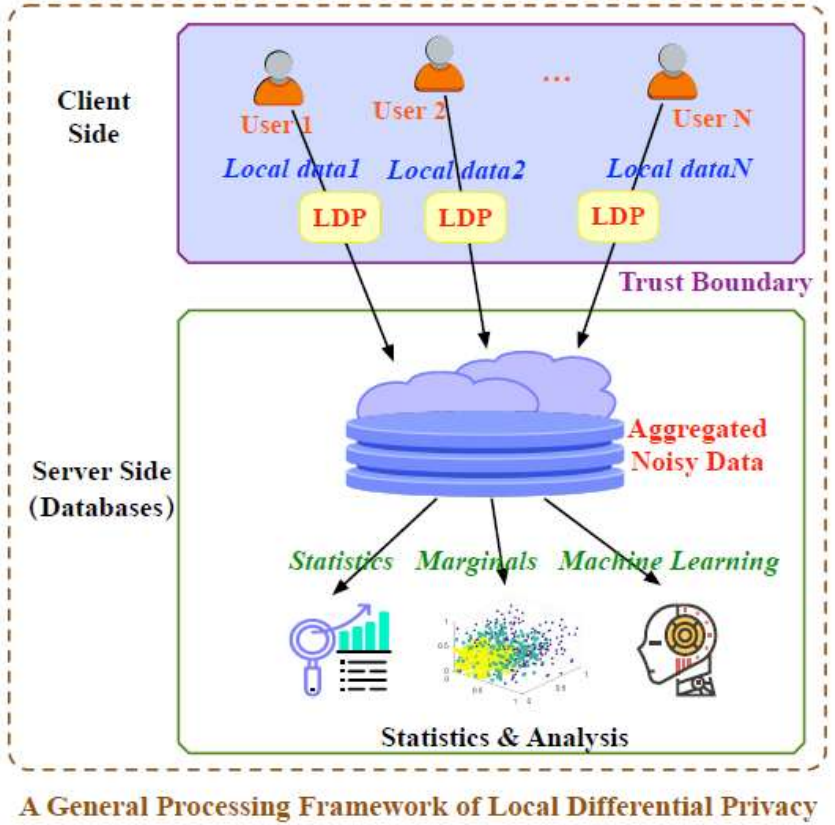
9.4数据库系统隐私性

➢ 本地化差分隐私

- ◆ 中心化差分隐私（传统差分隐私）：面向分析结果发布
- ◆ 本地化差分隐私（分布式差分隐私）：面向数据收集到分析全流程



A General Processing Framework of Differential Privacy



A General Processing Framework of Local Differential Privacy

9.4数据库系统隐私性

本地化差分隐私

随机化应答机制（Randomized Response）

S. L. Warner，1965，保护询问对象隐私的一种普查方法

- 例子：“你是否患有某种疾病？”
- 被调查者：
  - 投一枚硬币
  - 正面则回答事实
  - 反面则随机回答（是/否各50%概率）
- 因此，疾病患者回答“是”的概率是75%，回答“否”的概率是25%。



S. L. Warner

提供可否认性，即提供含有高度不确定性的敏感信息

9.4数据库系统隐私性

本地化差分隐私

随机化应答机制（Randomized Response）

- 观测：假设收到的回答是的总体比例为PY。
- 已知：疾病患者回答“是”的概率是75%，回答“否”的概率是25%。
  （同理，非患者回答“是”的概率为25%，回答“否”的概率为75%。）
- 未知：真实患病比例PA？　　（则非患病比例为1-PA）

- 求解：
  - 显然有，PY=PA*0.75+(1-PA)*0.25
  - 因此得，PA=(PY-0.25)/0.5

通过统计学推导，可得到高准确度的敏感数据无偏估计值。

9.4数据库系统隐私性

本地化差分隐私

本地化差分隐私的实践应用

- **Apple iOS**
  - **每天收集用户常用词、表情、健康数据、访问网页等数据，通过本地差分隐私机制处理后上传服务器。**
- **Google Chrome**
  - **在Chrome浏览器中每半小时收集上百个系统参数，通过本地差分隐私机制处理后上传服务器。**
- **Microsoft PINQ，Uber FLEX，Firefox, US Census Bureau, 小米 etc.**

